

Timer-Ersatzplatine — Projektplan

Stand: 2026-07-13 · Aktuelle Phase: **3 — Verifikation** [■] Pflegehinweis: Erledigtes abhaken ([x]), neue Erkenntnisse mit Datum ergänzen, niemals Messwerte ohne neue Messung ändern (siehe CLAUDE.md).

Phase 0 — Analyse & Vermessung abgeschlossen

- [x] Originalplatine fotografiert und Funktionsprinzip analysiert (Kondensatornetzteil, DS1302-RTC, 4-Digit-Display, Relais)
- [x] Bestückungsseiten-Scan 300 dpi → Außenmaße 76,7 × 66,1 (±0,3)
- [x] Lötseiten-Scan 300 dpi → Taster-Pinbilder, 4 Löcher, Aussparung
- [x] Gehäusescan → Registrierung über die 4 Rahmenschrauben (±0,4), Rahmenaußenkontur, 6 Dompositionen (±1,5) [■]
- [x] Höhenmessungen: 19,9 innen / 2 Rahmen / 16,5 über Platine
- [x] Erkenntnis: alte Aussparung = Freistellung Dom B5
- [x] Erkenntnis: 4 Befestigungsdome mit Stabilisierungstegen (fix!)

Phase 1 — Konzept abgeschlossen

- [x] Architektur: Taster → ESP32-C3 → PhotoMOS → Shelly-SW (Follow-Mode)
- [x] Timing WLAN-unabhängig, Konfiguration per ESPHome-Webserver
- [x] Shelly 1PM Mini Gen4 gewählt (29 × 34 × 16, 8 A) — Datenblatt liegt vor
- [x] Netzteil: TSP-05 (HLK-PM05-Klon, 5 V/3 W) — vorhanden
- [x] OLED 0,91" 128 × 32, rahmenmontiert, **gesteckt** auf J4
- [x] Alle Module liegen flach AUF der Platine (unten nur 1,8 mm!)

Phase 2 — Konstruktions- und Entwurfsvorlagen abgeschlossen

- [x] Parametrischer DXF-Generator `platine_template.py` (v8)
- [x] DXF mit Kontur, ø10-Durchführungen, Löchern, beschrifteten Stellflächen, Keepouts, J4/OLED-Steckposition [■]
- [x] KiCad-Schaltplan `timer_ersatzplatine.kicad_sch` (15 Bauteile, Netzlabels, Footprint-Vorbelegung)
- [x] ESPHome-Firmware `timer-relais-c3.yaml` (Pins synchron zum Plan)

Phase 3 — Verifikation am Objekt IN ARBEIT

Vor dem Einfrieren der Kontur zwingend abarbeiten:

- [] **Papier-Anprobe:** DXF/PDF 1:1 drucken, ausschneiden, ins Gehäuse legen — Dome, Taster, Fenster müssen treffen (fängt die ±1-1,5 mm ab)
- [] **Stegrichtungen** der 4 Befestigungsdome im Gehäuse ansehen und mit den RIBS-Annahmen im Template abgleichen (oben-links/rechts → Oberkante? Mitte-links → linker Rand? Mitte-rechts → Unterkante?)

- [] **Wandstärke** der Gehäuseschale messen (WALL_T, Annahme 2,0 mm)
- [] **TSP-05-Pinabstände** messen und gegen KiCad-Footprint HLK-PMxx vergleichen (AC-Pitch, DC-Pitch, Lage zur Kante)
- [] **OLED-Modul vermessen:** Linkskante→Pinreihe (Annahme 2,5), Linkskante→Zentrum aktive Fläche (Annahme 23,5)
- [] **Glaseinstand** messen → Stackhöhe J4 festlegen (~13–14 mm), Buchsen-/Stiftleisten-Kombination bestellen
- [x] **Dom-Positionen neu vermessen** (Silvio 13.07., Messschieber): Pitch 45 mm, Reihenabstand 70 mm, obere Linie 6 mm über Oberkante, linke Spalte 6 mm neben Linkskante → absolut verankert (x -6/39/84, y -6/+64); löst die Scan-Werte (y bis 2,6 mm daneben) ab
- [x] **Dom-Ausschnitte auf randoffene U-Schlitze umgestellt** (Breite 10, toleranter beim Einsetzen); Kontur bleibt geschlossene Edge.Cuts-Schleife (programmatisch verifiziert)
- [] **Dom-Durchmesser/-höhe** der 6 Gehäuseschrauben prüfen (Schlitzbreite 10 ausreichend? bei der Papier-Anprobe der neuen Kontur)
- [] Ggf. korrigierte Werte in platine_template.py eintragen, python3 platine_template.py ausführen, Anprobe wiederholen
- [] **Meilenstein: Kontur eingefroren** (Datum eintragen: __)

Phase 4 — KiCad-Layout offen

- [] Projekt anlegen, Schaltplan übernehmen, ERC (Doku Kap. 7.1)
- [] ESP32-C3-Super-Mini-Footprint anlegen (Doku Kap. 7.3)
- [] Footprints final zuordnen (Doku Kap. 7.2)
- [] DXF importieren, Kontur auf Edge.Cuts, Maßkontrolle T1→T2 = 25,6 mm
- [] 4 × MountingHole 3,2 mm auf Lochkoordinaten
- [] Taster + J4 numerisch exakt platzieren ($\pm 0,2!$)
- [] Module auf Stellflächen, 230-V-Ecke unten links aufbauen
- [] Netzklasse HV + 6-mm-Regel HV↔LV einrichten (Doku Kap. 7.6)
- [] Routing; O-Pfad ≥ 2 mm (8 A); DRC fehlerfrei
- [] Silkscreen: Stellflächen-Rahmen + Beschriftungen übernehmen
- [] 3D-Kontrolle, 1:1-Plot-Anprobe des fertigen Layouts

Phase 5 — Fertigung & Beschaffung offen

- [] Gerber + Bohrdaten exportieren, Bestellung (2 Lagen, 1,6 mm)
- [] Fehlende BOM-Teile bestellen: PhotoMOS ≥ 400 V, Buchsen-/Stift-leisten in gemessener Stackhöhe, Sicherungshalter, Varistor, Kleinteile (siehe Doku Kap. 3)

Phase 6 — Aufbau & Test offen

- [] Bestückung (flach → hoch), Taster exakt
- [] ESP vorab per USB flashen + WLAN-Test
- [] **Nur-USB-Funktionstest** (Taster, OLED, GPIO6) — ohne Netz!
- [] TSP einlöten, Shelly verkleben + verdrahten (L/N/SW/O)
- [] Sicht- & Durchgangsprüfung, Abstandskontrolle
- [] Erster Netztest über PRCD/RCD, Gehäuse geschlossen
- [] Shelly einrichten (WLAN, Input Mode „Switch/Follow“)

- [] Lasttest, Zeitentest, OTA-Update-Test

Phase 7 — Integration & Abschluss offen

- [] Zeiten final einstellen (<http://feeder-relais.local>)
- [] Optional: Einbindung in ioBroker (esphome-Adapter) / Shelly-Adapter
- [] Fotos des Aufbaus + „as built“-Abweichungen in die Doku
- [] Projektabschluss, Lessons Learned

Risiken & offene Punkte

#	Risiko	Auswirkung	Gegenmaßnahme	Status
1	Dom-Absolutlage noch $\sim \pm 1$ mm	Platine sitzt nicht satt über Dome	Positionen direkt vermessen; randoffene U-Schlitze fangen Rest-Streuung entlang der Achse; Papier-Anprobe	Schlitze erledigt, Anprobe offen
2	Stegrichtungen falsch angenommen	Modul kollidiert mit Steg	Sichtprüfung + Keepout-Korrektur	offen (Phase 3)
3	TSP-05 nicht HLK-pinkompatibel	Footprint falsch	Messschieber-Check vor Layout	offen (Phase 3)
4	OLED-Modulmaße streuen	Display sitzt schief im Fenster	Messung am realen Modul; J4 parametrisch	offen (Phase 3)
5	Stackhöhe passt nicht exakt	OLED drückt/hängt	Glaseinstand messen; Schaumstoffpad als Toleranzausgleich	offen (Phase 3)
6	Dom T3-Führung dicker als $\varnothing 8$	Shelly-Fläche kollidiert	bei Anprobe prüfen; Fläche kann 1–2 mm nach unten	offen (Phase 3)
7	Kriechstrecken im Layout	Sicherheit!	6-mm-DRC-Regel, Review vor Bestellung	Phase 4
8	Shelly-Wärme unter 0,5 mm Deckenluft	Derating	PM-Werte beobachten; Last ≤ 8 A ohnehin	Beobachtung

Entscheidungslog (warum ist etwas so?)

Datum	Entscheidung	Begründung
2026-07-13	Shelly schaltet, eigene Platine steuert nur SW	PM + Fernzugriff erhalten, Timing bleibt WLAN-unabhängig
2026-07-13	PhotoMOS ≥ 400 V statt Optokoppler/Relais	SW-Eingang ist netzspannungsbezogen; winzig; galvanische Trennung
2026-07-13	1PM Mini Gen4 statt 1PM Gen4	42 × 38 des großen passt flächig nicht
2026-07-13	Kein Versenken von Modulen	unter der Platine nur 1,8 mm (Dome hängen an der Front)
2026-07-13	TSP-05 statt HLK-PM05	vorhanden, gleiche Klasse; Stellflächen-Rochade ESP $\leftrightarrow$ PSU nötig
2026-07-13	OLED gesteckt statt verdrahtet	Montage/Service; J4 von Fensterzentrierung rückwärts konstruiert
2026-07-13	GPIO3/4/5/6 + 8/9	Strapping-Pins gemieden; 8/9 = SDA/SCL wie Modulaufdruck

Datum	Entscheidung	Begründung
2026-07-13	Dompositionen direkt per Messschieber statt aus Gehäusescan	Scan-y lag bis 2,6 mm daneben; Direktmaße (Pitch 45 / Reihen 70 / 6 mm Kantenabstand) sind genauer und lösen die $\pm 1,5$ -Registrierung ab
2026-07-13	Dom-Ausschnitte randoffen (U-Schlitz) statt $\varnothing 10$ -Bohrungen	toleranter beim Einsetzen, verzeiht Dom-Streuung entlang der Schlitzachse; äußere Eckradien dafür lokal auf 3,2/2,1 reduziert (spitzere Ecke passt mit mehr Spiel ins Gehäuse)
2026-07-13	Eigene Mobil-Web-App (timer_web.h) + JSON-API statt ESPHome-Standard-UI	mobile Bedien-/Einstell-/Netzwerk-/Statusseiten auf Port 80; eine /api/*-JSON-Schnittstelle bedient Handy und ioBroker gleich; mit esphome compile verifiziert
2026-07-13	Gerätename Default feeder-relais + NTP-Uhr, Tasten-Zustandsautomat, OLED-Redesign (Stufe A)	S1 Down/Manual · S2 SET · S3 UP: kurz=Timer, lang UP=Stop, lang SET=Info-Menü (10 s Timeout); OLED oben WLAN+Status, unten Uhr/ Countdown. esphome compile grün. Netzwerk-Laufzeitkonfig (WLAN/statisch/NTP/Hostname) = Stufe B
2026-07-13	Netzwerk-Laufzeitkonfig (Stufe B): Web-Formular + net_config.h	WLAN via save_wifi_sta, IP DHCP/statisch via ESP-IDF-netif bei wifi.on_connect, NTP via esp_sntp_setservername, persistent in Prefs, Neustart-Knopf. Hostname zunächst offen (siehe Folgezeile). esphome compile grün; Netzwerk-Anwendung braucht Gerätetest
2026-07-13	Hostname doch zur Laufzeit umbenennbar	ESPHome-mDNS nutzt ESP-IDF-mdns.h → mdns_hostname_set() stellt ...local sofort um; DHCP-Name über esp_netif_set_hostname() nach Reconnect/Neustart. Persistent, Web-Feld editierbar, DNS-Label-Sanitizer. esphome compile grün
2026-07-15	Beta-1-Fixes (Gerätetest)	(1) Kein Webserver im STA-Betrieb: web_server_base wird nur vom AP-Captive-Portal init()-siert → in on_boot selbst id(web_base) → init() aufgerufen (Port-80-Listener). (2) Schwankende Latenz/„mangelhaft“: wifi: power_save_mode: none. OLED separat: I2C Found no devices → Verdrahtung/Adresse prüfen (nicht firmwareseitig). esphome compile grün
2026-07-15	Kein einkompiliertes WLAN mehr – Provisioning nur per Captive-Portal	(1) kein fremder SSID/PW in der Auslieferung; (2) has_sta()==false → gespeicherte Portal-Daten unter festem Preference-Key 88491487 → überstehen OTA (mit einkompiliertem WLAN hängt der Key am Config-Hash → Verlust bei jedem Build, war der Beta-Bug). Factory = sauber (mit Flash-Erase), OTA behält WLAN. wifi: nur noch ap: + captive_portal. esphome compile grün
2026-07-16	I2C-SDA GPIO8 → GPIO7 ; Onboard-RGB-LED (GPIO8) als gedimmte Status-LED	GPIO8 trägt die WS2812 des Super Mini; der I2C-Verkehr trieb sie hell/warm. SDA daher auf GPIO7 (Schaltplan-Pin GPIO8/SDA → GPIO7/SDA inkl. Instanz-Nr., Doku/CLAUDE.md nachgezogen). WS2812 nun light: esp32_rmt_led_strip, im 1s-Interval: grün=OK / gelb=Timer / rot=Störung, ~25 %. Board-Übersicht + GPIO-Grafik in Doku Kap. 5.2. esphome compile grün
2026-07-13	Service-Tab: Live-Log/ Debug + Web-OTA + Neustart	logger: on_message: → Ringpuffer log_ring.h, GET /api/log gefiltert nach Stufe (ERROR...DEBUG); Web-Upload via ota: platform: web_server (POST /update, Port 80) neben Netzwerk-OTA (platform: esphome). esphome compile grün
2026-07-16	WLAN-Roaming 802.11k/v per Web schaltbar	Fähigkeit fest einkompiliert (wifi: enable_btm/enable_rrm: true → CONFIG_WPA_11KV_SUPPORT, nur so ist der set_btm/set_rrm-Codepfad aktiv). Ein/Aus zur Laufzeit über g_netcfg.roaming (NetCfg v2→v3, persistent): netcfg_apply_roaming() setzt set_btm/set_rrm + gegengleich set_post_connect_roaming (aus=ESPHome-Scan-Roaming). Web-

Datum	Entscheidung	Begründung
		Checkbox im Netzwerk-Tab, POST /api/net?roaming=0\ 1. Bits greifen ab nächstem (Re)Connect. Default aus . esphome compile grün
2026-07-16	Knopf „Jetzt neu verbinden“ (Netzwerk-Tab)	POST /api/reconnect setzt g_reconnect_pending; das 1s-Interval ruft dann netcfg_wifi_reconnect() = wifi.disable() + enable() (baut STA-Config neu → Roaming sofort aktiv, ohne Reboot). Verzögerung übers Interval, damit die HTTP-Antwort noch rausgeht (WLAN reißt danach kurz ab). Gleiches Muster wie g_reboot_pending. esphome compile grün
2026-07-16	Roaming-Haken persistent + sichtbar + Status-Zeile	Haken „verschwand“ nach Reconnect/Neustart. roaming jetzt in /api/status, der Haken wird laufend daraus gesetzt (self-healing, 4-s-Hold nach manueller Änderung) und bleibt sichtbar; neue Status-Zeile „WLAN-Roaming (802.11k/v): EIN/AUS“. esphome compile grün
2026-07-16	Panik-Neustarts behoben (Preferences aus Webserver-Task)	Ursache: netcfg_save() läuft im Webserver-Task; ESPHomes Preferences (s_pending_save-Vektor + NVS-Handle) sind ungelockt gegen den Main-Task. Der in v0.0.13 ergänzte global_preferences->sync() aus dem Handler = Data-Race → Heap-Korruption → ESP_RST_PANIC. Fix: netcfg_save() setzt nur g_prefs_save_pending; das 1s-Interval (Main-Task) ruft netcfg_flush() = save() + sync(). Alle Flash-Schreibzugriffe nun single-threaded. esphome compile grün
2026-07-16	Status: geflashte Version ganz oben	#define FW_VERSION in timer_web.h (bei jedem Release mitziehen), als ver in /api/status, erste Zeile „Version“ auf der Status-Seite. esphome compile grün
2026-07-16	Mehrsprachigkeit DE/EN/FR (Web + OLED)	Sprache geräteweit in g_netcfg.lang (NetCfg v3→v4, netcfg_lang_idx()), Auswahl per Dropdown ganz oben im Zeiten-Tab, POST /api/net?lang=de\ en\ fr. Web: JS-Wörterbuch T{key: {de,en,fr}} + data-i18n-Attribute, tr()/applyStatic(); sprachabhängige Statuswerte (WLAN/Reset) als neutrale Codes aus C++ (up/down, poweron/...), Übersetzung im Browser. OLED: Sprach-Tabellen für Wochentage/Status(Ruhe/Futter)/Menü, bewusst akzentfrei (Font-Glyphsatz). Logs + Captive-Portal bleiben englisch (ESPHome-Kern, nicht ohne Fork lokalisierbar). Doku: nur DOKUMENTATION.md nach EN+FR (DE führend, pro Release), Projektplan bleibt DE. esphome compile grün